# LifeSynch AI — Presentation, Viva & Demonstration Guide

## Comprehensive Guide for Project Viva, Live Demonstration, and Guide Review

**Project Title**: LifeSynch AI — Autonomous Academic & Personal Responsibility Hub
**Academic Level**: Minor Project, 3rd-Year B.Tech / B.E. (AIML / CSE / IT)

---

## Table of Contents


1. Elevator Pitch & 2-Minute Opening Speech
2. 5-Minute Full Presentation Script (Slide by Slide)
3. Click-by-Click Live Demonstration Script
4. 50 Comprehensive Viva Questions & Expert Answers
   - Category 1: Project Overview & Motivation (Q1 - Q6)
   - Category 2: System Architecture & Data Flow (Q7 - Q12)
   - Category 3: Google OAuth 2.0 & Token Security (Q13 - Q18)
   - Category 4: Gmail & Calendar API Integration (Q19 - Q24)
   - Category 5: Background Synchronization Engine (Q25 - Q29)
   - Category 6: AI Extraction & Gemini Integration (Q30 - Q35)
   - Category 7: Database Design & Supabase RLS (Q36 - Q41)
   - Category 8: Deadline, Timezone & Reminders Logic (Q42 - Q46)
   - Category 9: Testing, Limitations & Future Improvements (Q47 - Q52)
5. Live Demo Emergency Troubleshooting Guide


---

# 1. Elevator Pitch & 2-Minute Opening Speech

> *"Good morning, respected guide and external examiners.
> Today, a typical college student or professional receives dozens of emails and calendar invites every day across different platforms. Important assignment deadlines, exam announcements, and project milestones easily get buried under newsletters, spam, and promotional emails. Missing even one crucial deadline can cost students their grades or career opportunities.
> To solve this problem, we developed **LifeSynch AI** — an autonomous, intelligent responsibility hub that connects seamlessly with Google Workspace.
> Unlike a simple to-do list where users must manually type every task, LifeSynch AI automatically monitors Gmail and Google Calendar in the background every 30 seconds. Using advanced natural language processing powered by Google Gemini, it intelligently reads incoming messages, filters out promotional noise, separates actionable tasks from general announcements, extracts precise deadlines in Indian Standard Time, and categorizes them with priority rankings.
> Furthermore, users can converse with their schedule using our natural-language assistant, 'Ask LifeSynch AI', and receive real-time notifications for overdue, due-today, and upcoming tasks.
> In this presentation and live demonstration, we will show you how our system connects React, Express, Supabase PostgreSQL, and Google Gemini into a secure, cohesive product."*

---

# 2. 5-Minute Full Presentation Script (Slide by Slide)

## Slide 1: Title & Team

- **Spoken Words**: "*Our project is LifeSynch AI: Autonomous Academic & Personal Responsibility Hub. Our team consists of [Your Name] and [Partner Names]. Guided by [Guide Name].*"

## Slide 2: Problem Statement & Existing Gaps

- **Spoken Words**: "*Existing solutions like Google Tasks, Todoist, or Notion require 100% manual entry. If an email arrives stating 'DBMS Lab Assignment 3 due next Tuesday at 4 PM', the user must manually copy it over. In reality, students forget to do this. LifeSynch AI removes manual friction by automating extraction at the source.*"

## Slide 3: Objectives & Key Features

- **Spoken Words**: "*Our core objectives were: first, zero-friction Google OAuth integration; second, silent background synchronization every 30 seconds; third, AI-powered extraction of tasks and deadlines; fourth, intelligent separation of actionable tasks from 'Worth Knowing' announcements; and fifth, an offline-resilient conversational assistant.*"

## Slide 4: System Architecture

- **Spoken Words**: "*As shown in our architecture diagram, the frontend is built using React with TypeScript and styled with Tailwind CSS. The backend is an Express Node.js server. Data is stored in Supabase PostgreSQL with strict Row Level Security (RLS). External services include Google Gemini for LLM intelligence and Google Cloud for Gmail and Calendar APIs.*"

### Slide 5: Database Design & Security

- **Spoken Words**: "*We designed seven relational tables: Users, Connected Accounts, Emails, Calendar Events, Responsibilities, Notifications, and Sync State. Each table enforces user isolation using PostgreSQL Row Level Security policies, ensuring no user can ever access another user's private data.*"

### Slide 6: Results & Live Verification

- **Spoken Words**: "*All frontend and backend builds compile cleanly. Our comprehensive automated test suites for security, OAuth, Gmail, Calendar, reminders, and conversational assistant pass with 100% success. We are now ready to demonstrate the live application.*"

---

## 3. Click-by-Click Live Demonstration Script

Follow these exact steps during your demonstration to showcase the system smoothly:

| Step | Action on Screen | What to Say to the Examiner | Behind-the-Scenes Mechanism |
|---|---|---|---|
| **1** | Open browser to `http://localhost:5173`. Show the landing / login screen. | *"Here is the landing page. Notice that we don't ask users to create a separate password. We use standard Google OAuth 2.0."* | Frontend initiates OAuth flow with redirect URI pointing to Express backend. |
| **2** | Click **"Sign in with Google"** and complete Google consent. | *"Google prompts for read-only access to Gmail and Calendar. Notice we request minimal read-only permissions."* | Google issues authorization code; backend exchanges it for `access_token` and `refresh_token` stored in `connected_accounts`. |
| **3** | Lands on **Dashboard**. Point out the **Sync Status Pill** in top right. | *"We are now on the Dashboard. Look at this status badge: it shows 'Synced just now'. The background service runs every 30 seconds automatically."* | `AutoSyncService` timer running in backend; writes status to `sync_state` table; frontend polls sync status endpoint. |
| **4** | Point to the **Quick Stats Cards** (Total Tasks, Due Today, High Priority, Completed). | *"The dashboard provides immediate situational awareness. These counts reflect live records from PostgreSQL."* | Aggregate SQL queries run through Supabase client with user ID filter. |
| **5** | Scroll to the **"Actionable Responsibilities"** list. Point out a task card. | *"Here is an assignment extracted from an email. Notice the title, category badge ('Academic'), priority badge ('High'), and exact deadline."* | Gemini extracted the task fields, verified by Zod schema, stored in `responsibilities` table. |
| **6** | Click on the **"Worth Knowing"** tab or card. | *"Here is our Worth Knowing section. For example, 'College Sports Day next week' is an announcement, not a task with a deliverable. Our AI categorizes it separately so it doesn't clutter your task list."* | Flagged with `itemType: "worth_knowing"` in JSONB metadata. |
| **7** | Click the **Checkmark / Complete** button on a task. | *"When I mark a task complete, notice it immediately updates to completed state with a strikethrough."* | `PATCH /api/responsibilities/:id` sets `status = 'completed'` and `completed_at = now()`. |
| **8** | Click the **Notification Bell** icon in the header. | *"Notice the notifications dropdown. The completed task is immediately removed from reminders, leaving only active pending deadlines."* | `ReminderService` queries pending tasks and excludes completed/cancelled items dynamically. |
| **9** | Open the **"Ask LifeSynch AI"** chat drawer. Type: *"What tasks do I have due this week?"* | *"Now we interact with the AI assistant. Even under free-tier quota constraints, our grounded rule-based fallback ensures reliable, deterministic answers directly from user data."* | Assistant detects `this_week` intent and returns filtered responsibilities with formatting. |

# 4. 50 Comprehensive Viva Questions & Expert Answers

## Category 1: Project Overview & Motivation (Q1 - Q6)

**Q1: What is the main objective of LifeSynch AI?**

**Answer**: The main objective is to provide an autonomous responsibility management system that automatically converts unstructured emails from Gmail and events from Google Calendar into structured, prioritized tasks with deadlines, eliminating manual data entry.

**Q2: How is LifeSynch AI different from existing tools like Google Tasks or Todoist?**

**Answer**: Google Tasks and Todoist require users to manually type every task and set dates themselves. LifeSynch AI acts as an autonomous intelligence layer that automatically ingests messages, extracts deadlines via AI, categorizes priorities, and organizes announcements into a "Worth Knowing" feed without manual input.

**Q3: Who is the target user for this application?**

**Answer**: University students, researchers, faculty, and busy professionals who receive high volumes of academic notices, project deadlines, meeting schedules, and general announcements.

**Q4: Why did you choose Google Workspace (Gmail & Calendar) as the primary data sources?**

**Answer**: Google Workspace is the predominant email and scheduling platform used by universities and organizations worldwide, offering well-documented REST APIs with OAuth 2.0 security.

**Q5: What are the three core functional pillars of LifeSynch AI?**

**Answer**:

1. *Ingestion & Synchronization*: Background fetching of Gmail and Calendar data.
2. *Intelligent Understanding*: AI extraction of tasks, priorities, deadlines, and informational notices.
3. *Actionable Presentation*: Dashboard, notification system, and conversational assistant.

**Q6: What was the motivation behind having a "Worth Knowing" section?**

**Answer**: Not every important email contains a direct action item. For example, an email stating "Campus Wi-Fi will be down this Saturday for maintenance" is vital to know, but does not belong on a student's to-do checklist. Separating them prevents task list fatigue while preserving awareness.

---

## Category 2: System Architecture & Data Flow (Q7 - Q12)

**Q7: Can you describe the overall system architecture?**

**Answer**: We use a 3-tier architecture:

- *Presentation Tier*: React SPA built with Vite and Tailwind CSS.
- *Application Tier*: Node.js and Express REST API backend managing OAuth, sync services, AI pipelines, and reminders.
- *Data Tier*: Supabase PostgreSQL cloud database with Row Level Security (RLS), alongside external Google APIs and Google Gemini LLM.

#### Q8: How does the frontend communicate with the backend?

**Answer**: The frontend makes standard asynchronous HTTP calls (using `fetch` or Axios) to backend REST API endpoints prefixed with `/api/*`. Requests include session authentication cookies or Bearer tokens.

#### Q9: What is the role of the `shared/` workspace in your project?

**Answer**: The `shared/` workspace contains shared TypeScript interfaces, type definitions, and schema constants imported by both frontend and backend. This guarantees single-source-of-truth type safety and prevents interface drift.

#### Q10: How does data flow from an incoming email to the dashboard?

**Answer**:

1. Gmail API delivers raw email JSON.
2. Ingestion service stores raw email in `emails` table.
3. AI extraction service sends email text to Gemini LLM with prompt and schema.
4. Gemini returns structured JSON; backend validates it with Zod.
5. Record is inserted into `responsibilities` table.
6. Frontend queries `/api/responsibilities` and renders the task card.

#### Q11: Why did you choose Express over other frameworks like NestJS or Fastify?

**Answer**: Express is lightweight, transparent, highly reliable, and widely understood. It allowed us to implement custom middleware, background timers, and OAuth pipelines with minimal framework overhead.

#### Q12: How does the system handle CORS (Cross-Origin Resource Sharing)?

**Answer**: The backend uses the `cors` middleware configured with `origin: 'http://localhost:5173'` and `credentials: true`, allowing secure browser communication between the Vite dev server and Express backend.

---

## Category 3: Google OAuth 2.0 & Token Security (Q13 - Q18)

#### Q13: What is OAuth 2.0 and why did you use it?

**Answer**: OAuth 2.0 is an industry-standard authorization framework. We use it so users can grant LifeSynch AI access to their Gmail and Calendar data without ever giving us their Google password.

#### Q14: What is the difference between an `access_token` and a `refresh_token`?

**Answer**: An `access_token` is a short-lived credential (valid for 1 hour) sent with API requests to Google. A `refresh_token` is a long-lived credential stored in our database, used to securely obtain a new `access_token` when the existing one expires without prompting the user.

#### Q15: Where are user tokens stored in the database?

**Answer**: In the `connected_accounts` table, linked to the user's primary key (`user_id`). The table stores `access_token`, `refresh_token`, `token_expiry`, and granted scopes.

#### Q16: What Google OAuth scopes does LifeSynch AI request?

**Answer**:

- `openid`, `email`, `profile` (for user authentication)

- `https://www.googleapis.com/auth/gmail.readonly` (read-only email access)
- `https://www.googleapis.com/auth/calendar.readonly` (read-only calendar access)

**Q17: Why did you request read-only scopes instead of full access?**

**Answer**: Principle of Least Privilege: LifeSynch AI only needs to read emails and events to extract tasks. It does not need permission to send emails, delete messages, or alter calendar events, minimizing security risks.

**Q18: What happens if a user revokes access from their Google Account settings?**

**Answer**: Subsequent API calls return HTTP 401 Unauthorized. The backend catches this error, marks the account status as disconnected, and prompts the user on the frontend to reconnect.

---

## Category 4: Gmail & Calendar API Integration (Q19 - Q24)

**Q19: How do you prevent fetching the same email repeatedly?**

**Answer**:

1. We query Gmail with a time filter ( `newer_than:7d` ).
2. The database has a unique constraint on `(user_id, message_id)` in the `emails` table. Upsert operations ensure duplicate emails are ignored.

**Q20: How do you extract the plain text body from a MIME multi-part email?**

**Answer**: Gmail messages are nested MIME trees encoded in base64. Our `extractEmailBody` utility recursively traverses `payload.parts`, decodes `text/plain` or strips HTML from `text/html`, and decodes standard URL-safe base64.

**Q21: How does Calendar event fetching work?**

**Answer**: We call `calendar.events.list` for the `'primary'` calendar, filtering events from 7 days in the past to 30 days in the future ( `timeMin` to `timeMax` ) with `singleEvents: true` to expand recurring events.

**Q22: How does the system handle cancelled Google Calendar events?**

**Answer**: Google Calendar API marks deleted events with `status: 'cancelled'`. Our sync service detects this flag, updates the corresponding record in `calendar_events`, and marks associated responsibilities as cancelled.

**Q23: What is the difference between an all-day event and an exact-time event?**

**Answer**:

- *All-day event*: Has only `start.date` (e.g., `"2026-11-10"` ), representing a full day.
- *Exact-time event*: Has `start.dateTime` (e.g., `"2026-11-10T10:00:00+05:30"` ), representing a specific start and end time.

**Q24: How are email attachments handled?**

**Answer**: For this minor project scope, attachment metadata is noted, but attachment binary downloads are skipped to optimize processing speed and stay within storage boundaries.

---

## Category 5: Background Synchronization Engine (Q25 - Q29)

### Q25: How does automatic synchronization work in LifeSynch AI?

**Answer**: `AutoSyncService` runs a Node.js recurring timer every 30,000 milliseconds (30 seconds). On each tick, it iterates over all active connected accounts and synchronizes both Gmail and Calendar.

### Q26: How do you prevent overlapping synchronization cycles?

**Answer**: We use an in-memory boolean flag `isRunning`. If a sync cycle is still executing when the 30-second timer fires, the new cycle exits immediately, acting as a lightweight mutex lock.

### Q27: What happens if Calendar sync fails due to network issues? Does Gmail sync also fail?

**Answer**: No. We execute both synchronization jobs inside `Promise.allSettled()`. If Calendar sync fails, its error is logged independently, while Gmail sync completes and commits its data normally.

### Q28: How does the frontend know when data was last synchronized?

**Answer**: The backend writes sync timestamps and status (`idle`, `syncing`, `error`) to the `sync_state` table. The frontend polls `/api/sync/status` and displays a dynamic badge like "Synced 2 mins ago".

### Q29: Can the user manually trigger a sync?

**Answer**: Yes, the frontend includes a manual "Sync Now" button that sends a `POST /api/sync/trigger` request to invoke `AutoSyncService.syncUser()` on demand.

---

## Category 6: AI Extraction & Gemini Integration (Q30 - Q35)

### Q30: Which AI model is used in LifeSynch AI?

**Answer**: We use Google's **Gemini Flash** model via the official `@google/genai` SDK, configured with the environment variable `LLM_API_KEY`.

### Q31: How do you ensure the LLM output is structured and valid?

**Answer**: We supply a system prompt requiring pure JSON output conforming to a defined schema, and then validate the parsed JSON at runtime using a TypeScript **Zod** schema (`z.object({...})`).

### Q32: What happens if Gemini API key quota is exhausted (HTTP 429)?

**Answer**: Google Free Tier has a strict quota of 20 requests per day. When quota exhaustion occurs, the system logs the 429 error gracefully. For conversational queries, our system automatically activates a **deterministic grounded fallback** that answers queries using direct database queries.

### Q33: How does the AI determine the priority of a task?

**Answer**: The prompt instructs the model to evaluate urgency keywords (e.g., "urgent", "mandatory", "by tonight", "exam", "penalty") and deadline proximity. Tasks with immediate deadlines or severe consequences are categorized as `high`, standard deliverables as `medium`, and optional tasks as `low`.

### Q34: What categories are supported for responsibilities?

**Answer**: Four distinct categories: `academic`, `work`, `personal`, and `financial`.

**Q35: How does the AI extract deadlines from informal language like "next Friday afternoon"?**

**Answer**: The prompt passes the current reference timestamp (`currentIso`) to Gemini. The LLM calculates the relative offset (e.g., "next Friday" relative to today's date) and resolves it to a concrete ISO date and time.

---

## Category 7: Database Design & Supabase RLS (Q36 - Q41)

**Q36: Name all 7 tables in your Supabase database.**

**Answer**:

1. `users` — Registered user profiles.
2. `connected_accounts` — Google OAuth tokens and provider details.
3. `emails` — Ingested Gmail messages.
4. `calendar_events` — Ingested Google Calendar events.
5. `responsibilities` — Extracted tasks and announcements.
6. `notifications` — Generated reminders and alert history.
7. `sync_state` — Background sync status and timestamps.

**Q37: Where is the clock time of a deadline stored in the `responsibilities` table?**

**Answer**: The date is stored in the `due_date` column (`TIMESTAMPTZ`), and the specific clock time is stored in the `metadata` JSONB column as `metadata.dueTime` (e.g., `{"dueTime": "14:30"}`).

**Q38: Why did you use a JSONB `metadata` column instead of adding 10 different columns?**

**Answer**: JSONB provides flexibility for semi-structured data like source references, confidence scores, extraction rationale, and `dueTime`, allowing schema evolution without requiring table migrations.

**Q39: What is Row Level Security (RLS)?**

**Answer**: Row Level Security is a PostgreSQL security feature that evaluates SQL policies on every query. If User A logs in, PostgreSQL will only return rows where `user_id = auth.uid()`, preventing cross-tenant data leaks.

**Q40: What are foreign key constraints used in your database?**

**Answer**: Every secondary table (`emails`, `calendar_events`, `responsibilities`, etc.) references `users(id)` with `ON DELETE CASCADE`. If a user account is deleted, all their associated emails, events, and tasks are automatically purged.

**Q41: How do you handle database migrations?**

**Answer**: SQL migration scripts are maintained in `backend/supabase/migrations/` and applied sequentially to ensure version-controlled schema parity.

---

## Category 8: Deadline, Timezone & Reminders Logic (Q42 - Q46)

**Q42: How does LifeSynch AI handle Indian Standard Time (IST)?**

**Answer**: Indian Standard Time is UTC+05:30. When parsing date-only strings (like all-day calendar events), the system explicitly appends `+05:30` and sets the timestamp to `23:59:59` IST so that the task remains due at the end of the user's day rather than shifting into the previous day.

**Q43: How does the ReminderService classify notifications?**

**Answer**: It scans pending responsibilities and places them into three categories:

- *Overdue*: Deadline timestamp < current time.
- *Due Today*: Deadline falls between 00:00:00 and 23:59:59 IST of today.
- *Upcoming*: Deadline is within the next 48 hours.

**Q44: Are completed or cancelled responsibilities alerted as reminders?**

**Answer**: No. `ReminderService` strictly filters for `status = 'pending'`. Completed, archived, or cancelled items are excluded.

**Q45: How does the notification display test verify dynamic dates?**

**Answer**: We wrote dynamic test suites that calculate timestamps relative to `Date.now()` (e.g., `+2 hours`, `-1 hour`, `+24 hours`) rather than hardcoded calendar dates, ensuring tests never expire.

**Q46: Can a user customize reminder lead times?**

**Answer**: In the current version, defaults are 24 hours and morning-of-due-date. User-customizable notification lead times are planned for the major project phase.

---

## Category 9: Testing, Limitations & Future Improvements (Q47 - Q52)

**Q47: What test suites did you run to verify the project?**

**Answer**:

1. `test:security` — Verifies token isolation and authentication middleware.
2. `test:gmail` — Tests MIME decoding, message fetching, and deduplication.
3. `test:calendar` — Tests event parsing, cancelled events, and all-day handling.
4. `test:auto-sync` — Tests 30-second loop, concurrency locks, and error recovery.
5. `test:assistant` — Tests intent parsing and conversational grounded fallbacks (21/21 passed).
6. `test:reminders` — Tests timezone parsing, deadline buckets, and exclusions.

**Q48: What is the current verification status of live Gemini extraction?**

**Answer**: The live Gemini API call is marked as **NOT VERIFIED** due to Google Free Tier quota exhaustion (HTTP 429, limit of 20 requests per day). However, all supporting pipeline components, schema validators, and deterministic fallbacks are fully verified and operational.

**Q49: What happens if the network drops while the app is running?**

**Answer**: The frontend displays a connection warning banner. The background sync logs a network timeout and retries automatically on the next 30-second cycle without crashing the server.

**Q50: What are the main limitations of the current minor project?**

**Answer**:

1. Only Google Workspace (Gmail/Calendar) is supported; Outlook and Apple Calendar are not yet integrated.
2. Google Gemini Free Tier limits live extraction requests to 20 per day.
3. Push notifications are currently in-app only; SMS or WhatsApp alerts are not yet connected.

**Q51: What are your planned enhancements for the Major Project?**

**Answer**:

1. Integration with Microsoft Graph API (Outlook and Teams).
2. WhatsApp and Telegram notification bot.
3. On-device lightweight open-source LLM (such as LLaMA 3 or Mistral via Ollama) for zero-cost, unlimited local extraction.
4. Canvas / Moodle LMS API connectors for direct academic coursework ingestion.

**Q52: What is the single most valuable lesson you learned building this project?**

**Answer**: We learned that integrating real-world third-party APIs requires resilient defensive programming: handling token refreshes, managing timezone conversions across UTC and IST, validating unpredictable LLM outputs with Zod schemas, and providing deterministic fallbacks when external APIs hit quota limits.

---

## 5. Live Demo Emergency Troubleshooting Guide

If something unexpected happens during your live guide demonstration, stay calm and use these immediate solutions:

| Symptom During Demo | Probable Cause | Immediate Action to Take |
|---|---|---|
| **Google Sign-in redirects to an error page** | Google OAuth token expired or port mismatch. | 1. Check if backend terminal is running on port 5000.<br>2. Clear browser cookies for `localhost` and retry.<br>3. Verify redirect URI matches Google Cloud Console. |
| **Tasks do not appear after sync** | Gemini API free quota exhausted (HTTP 429). | Explain honestly: "*The live LLM hit the Google Free Tier quota limit of 20 requests/day. Notice how our system gracefully handles this by preserving existing tasks and activating our grounded conversational fallback.*" |
| **Frontend displays a white screen** | Vite dev server lost connection. | Press `Ctrl + R` (hard refresh) or restart `npm run dev -w frontend`. |
| **Sync status says "Error"** | Internet disconnection or Google API timeout. | Click the manual "Sync Now" button. The server will re-attempt connection. |
| **Examiner asks to show database records** | Verification of real database integration. | Open Supabase Studio in the browser ( `https://app.supabase.com` ), navigate to Table Editor, and show the live rows in `responsibilities` and `emails`. |